# Digital Wallet & Payment Platform — Full Engineering Assignment

> ***Inspired by:*** *Venmo / PayPal* ***Target learner:*** *3rd-year computer science student who has completed a full-stack assignment (REST API, auth, basic UI)* ***Goal:*** *Build a production-grade payment platform from a blank repository across ten phases, each introducing a manageable layer of new complexity.* ***Cloud Budget:*** *≤ $20 USD/month on AWS (Free Tier maximised; expensive managed services replaced with self-hosted Docker equivalents)*

---

## How Progressive Overload Works Here

Each phase introduces **one primary new concept cluster** — not five. You build something that works, see its limitation, and the next phase fixes that limitation. Nothing is introduced and immediately discarded.

Rules:

- Complete phases in order. Every phase builds on the previous one.
- Each phase ends with a **Milestone Test** — a concrete automated check. Do not move on until it passes.
- Write a **README update** after every phase explaining your decisions.
- All code is written by you. You must be able to explain every line.

---

## Cross-Cutting Standards (Phase 1 onward, forever)

These apply everywhere and are not repeated per phase.

### Git

- One commit per logical change. Imperative mood: `Add idempotency key to transactions table`.
- `.gitignore` must cover: `node_modules/`, `.env`, `*.pem`, `*.key`, `terraform.tfstate`, `terraform.tfstate.backup`, `.terraform/`.
- Never commit secrets. If you do accidentally, rotate the secret — rewriting history does not help after a push.

### TypeScript

- Use TypeScript with `"strict": true` in `tsconfig.json` for all backend and frontend code.
- No `any` without a comment explaining why it cannot be avoided.
- ESLint + Prettier configured from Phase 1.

### Logging

Structured JSON to stdout from every service:

```
{ "level": "info", "msg": "Transfer completed", "service": "wallet-service", "requestId": "abc-123",
"ts": "2026-09-11T10:00:00Z" }
```

Never log passwords, tokens, or PII. Levels: `error` (exceptions), `warn` (degraded), `info` (normal), `debug` (off in production).

### Error Responses

```
{
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "Your wallet balance is too low for this transfer.",
    "requestId": "abc-123"
```

```
  }
}
```

### API Documentation

Every REST endpoint documented in OpenAPI 3.0. Swagger UI served at `/docs` in development.

---

## Technology Defaults

| Concern | Technology |
|---|---|
| Backend | Node.js (Express) + TypeScript |
| Frontend | React + TypeScript + Tailwind CSS |
| Database | PostgreSQL 15 |
| Cache | Redis 7 |
| Container | Docker + Docker Compose |
| CI/CD | GitHub Actions |
| Cloud | AWS (Free Tier first) |
| IaC | Terraform >= 1.6 |
| Testing | Jest (unit) + shell scripts (integration) |

---

# Phase 0 — Prerequisites & Environment Setup

**Objective:** Confirm you have the foundational knowledge and tools required before writing a line of this project. This phase has no code — only verification.

**Estimated Time:** 4–6 hours (learning gaps) or 1 hour (if fully prepared) **Cost:** $0

---

## Knowledge Checklist

Work through any item you cannot answer confidently before starting Phase 1.

### HTTP & REST

- What is the difference between `GET`, `POST`, `PUT`, `PATCH`, `DELETE`?
- What does a `401` vs `403` status code mean? When would you return each?
- What are HTTP headers? What is `Content-Type` and `Authorization`?
- What is CORS and why does a browser block cross-origin requests?

401 Unauthorized means the request has not been successfully authenticated, such as when the access token is missing, invalid, or expired. 403 Forbidden means the user is authenticated, but doesn't have sufficient permissions to access the requested resource

**If unclear:** MDN HTTP Overview

### SQL & PostgreSQL

- What is a `FOREIGN KEY`? What does `ON DELETE CASCADE` do?
- What is the difference between a `JOIN` and a subquery?
- What is a database transaction? What does `BEGIN` / `COMMIT` / `ROLLBACK` do?
- What is an `INDEX` and when should you add one?

**If unclear:** PostgreSQL Tutorial — complete sections 1–6

### JavaScript / Node.js

- What is `async/await`? What is a `Promise`? What happens if you forget `await`?
- What is `try/catch/finally`? When does `finally` run?
- What is a module? How does `import` / `export` work?

**If unclear:** javascript.info — Async

### TypeScript

- What is a type annotation? What is an `interface` vs a `type`?
- What does `strict: true` do? What is `noImplicitAny`?
- What is a generic (e.g., `Array<T>` or `Promise<string>`)?

**If unclear:** TypeScript Handbook — first 5 sections

### Authentication Basics

- What is a session cookie? How does a server know who you are between requests?
- What is a JWT? What are its three parts (header, payload, signature)?
- What is the difference between authentication (who are you?) and authorisation (what can you do?)?

**If unclear:** JWT Introduction

---

## Tools Installation Checklist

Verify each is installed and working:

```
node --version        # >= 20.0.0
npm --version         # >= 10.0.0
docker --version      # >= 24.0.0
docker compose version # >= 2.20.0
psql --version        # >= 15 (or use Docker — see below)
git --version         # >= 2.40.0
```

If you do not have PostgreSQL installed locally, you do not need to — Phase 1 uses Docker Compose to run it. But you must have Docker.

### Verify Docker works:

```
docker run --rm hello-world
```

If that outputs "Hello from Docker!", you are ready.

---

## Starter Project Setup

```
mkdir digital-wallet
cd digital-wallet
git init
echo "node_modules/\n.env\n*.pem\n*.key\nterraform.tfstate\nterraform.tfstate.backup\n.terraform/" >
.gitignore
git add .gitignore
git commit -m "Initial commit"
```

Create accounts you will need:

- GitHub — for version control and CI/CD
- AWS — for cloud deployment (Phase 6). Use the Free Tier.

---

### Milestone Test — Phase 0

Answer these questions in writing (add them to `docs/learning-log.md`):

1. A user sends a POST request to `/transactions/send` without a JWT. What HTTP status code do you return and why?
2. Two users both try to send money from the same wallet at exactly the same time. What could go wrong? (You will solve this properly in Phase 5a — for now, just describe the problem.)
3. What is the difference between `NUMERIC(15,2)` and `FLOAT` in PostgreSQL? Which one would you use to store a dollar amount and why?
4. Describe in one paragraph what a JWT is and how a server uses it to verify a request.

If you cannot answer all four clearly, revisit the relevant Knowledge Checklist section above.

---

---

# Phase 1 — Foundation: Monolithic Wallet App

**Objective:** Build a working full-stack digital wallet as a single application. This is your baseline — every later phase refactors or extends it.

**Estimated Time:** 10–14 hours **Cost:** $0 (runs entirely on your local machine)

---

### Before You Start This Phase

**You must already know:** REST API basics, SQL transactions, `async/await`, basic TypeScript, what a JWT is.

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| bcrypt | Password hashing algorithm with a configurable work factor (makes brute force slow) | auth0.com/blog/hashing-passwords |
| JWT signing (HS256) | Signing a token with a shared secret so it cannot be forged | jwt.io/introduction |
| Refresh tokens | Short-lived access token + long-lived refresh token pattern | auth0.com/blog/refresh-tokens |
| Redis (key-value cache) | In-memory store for fast lookups | redis.io/docs/get-started — sections 1–3 |
| httpOnly cookie | A browser cookie JavaScript cannot read — used to store refresh tokens safely | MDN Set-Cookie |
| Input validation (zod) | Schema-based validation library for TypeScript | zod.dev |

Spend 2–3 hours on these before writing code.

---

### What You Will Build

A web application where users can:

- Register and log in
- View their wallet balance

- Deposit funds (simulated — no real payment gateway)
- Send money to another registered user by email
- View transaction history, paginated

---

## Step 1: Project Structure

```
digital-wallet/
├── backend/
│   ├── src/
│   │   ├── routes/
│   │   ├── controllers/
│   │   ├── services/
│   │   ├── middleware/
│   │   ├── models/
│   │   └── app.ts
│   ├── migrations/          ← numbered SQL files: 001_init.sql
│   ├── tsconfig.json
│   ├── package.json
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   └── main.tsx
│   └── package.json
├── docker-compose.yml        ← runs PostgreSQL and Redis locally
├── docs/
│   └── learning-log.md
└── README.md
```

`docker-compose.yml` at this phase only runs PostgreSQL and Redis — the backend and frontend still run with `npm run dev` on your machine.

---

## Step 2: Database Schema

```sql
-- 001_init.sql

CREATE TABLE schema_migrations (
  version    TEXT PRIMARY KEY,
  applied_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE users (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  full_name     TEXT NOT NULL,
  locked_until  TIMESTAMPTZ,
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE refresh_tokens (
```

```
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    UUID REFERENCES users(id) ON DELETE CASCADE,
  token_hash TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  revoked    BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE wallets (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,
  balance    NUMERIC(15, 2) NOT NULL DEFAULT 0.00 CHECK (balance >= 0),
  currency   TEXT NOT NULL DEFAULT 'USD',
  frozen     BOOLEAN NOT NULL DEFAULT FALSE,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Auto-update updated_at whenever balance changes
CREATE OR REPLACE FUNCTION update_wallet_timestamp()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER wallet_updated_at
  BEFORE UPDATE ON wallets
  FOR EACH ROW EXECUTE FUNCTION update_wallet_timestamp();

CREATE TABLE transactions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sender_id       UUID REFERENCES users(id),      -- NULL for deposits
  receiver_id     UUID REFERENCES users(id) NOT NULL,
  amount          NUMERIC(15, 2) NOT NULL CHECK (amount > 0),
  type            TEXT NOT NULL CHECK (type IN ('deposit', 'transfer')),
  status          TEXT NOT NULL DEFAULT 'pending'
                  CHECK (status IN ('pending', 'completed', 'failed')),
  note            TEXT,
  idempotency_key UUID UNIQUE,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_tx_sender   ON transactions(sender_id, created_at DESC);
CREATE INDEX idx_tx_receiver ON transactions(receiver_id, created_at DESC);
```

**Why `NUMERIC(15,2)` not `FLOAT`:** Floating point numbers cannot represent 0.1 exactly in binary. `0.1 + 0.2` in IEEE 754 is `0.30000000000000004`. For money, you must use fixed-point arithmetic. Explain this in your README.

**Why the trigger on `wallets`:** PostgreSQL does not auto-update `updated_at`. Without the trigger, the column never changes regardless of how many times you update the balance.

---

## Step 3: Authentication — HS256 First

> ***Note:*** *This phase uses HS256 (shared secret) for simplicity. Phase 2c upgrades to RS256 (asymmetric) when you need multiple services to verify tokens without sharing a secret. Learn the simpler version first, understand why it breaks in a distributed system, then upgrade.*

### Endpoints

| Method | Path | Description |
|---|---|---|
| POST | `/auth/register` | Hash password, create user + wallet, return user |
| POST | `/auth/login` | Verify password, return access token (body) + refresh token (httpOnly cookie) |
| POST | `/auth/refresh` | Verify refresh cookie, issue new access token |
| POST | `/auth/logout` | Revoke refresh token in DB |

### Password hashing

```
import bcrypt from 'bcrypt';
const BCRYPT_ROUNDS = 12;
const hash = await bcrypt.hash(password, BCRYPT_ROUNDS);
const match = await bcrypt.compare(plaintext, hash);
```

Cost factor 12 means each hash attempt takes ~250ms on a modern CPU — acceptable for a login endpoint but impractical for an attacker trying millions of passwords per second.

### JWT (HS256)

```
import jwt from 'jsonwebtoken';
const accessToken = jwt.sign({ userId, email }, process.env.JWT_SECRET, { expiresIn: '15m' });
```

Store `JWT_SECRET` in `.env`. Never hardcode it. Never commit `.env`.

### Refresh token storage

Do not store the raw refresh token. Store its SHA-256 hash:

```
import crypto from 'crypto';
const tokenHash = crypto.createHash('sha256').update(refreshToken).digest('hex');
// INSERT INTO refresh_tokens (user_id, token_hash, expires_at) VALUES (...)
```

On refresh: hash the incoming cookie value, look it up in the DB, check it is not revoked or expired.

### Brute-force protection on login

```
// Before checking the password:
const user = await getUserByEmail(email);
if (user.locked_until && user.locked_until > new Date()) {
  throw new AppError('ACCOUNT_LOCKED', 'Too many failed attempts. Try again later.', 429);
}

// After a failed password check:
await incrementFailureCount(email);   // if >= 5, set locked_until = NOW() + 15 minutes

// After a successful login:
await clearFailureCount(email);
```

Store the failure counter in Redis ( `login-failures:{email}` , TTL 15 minutes) — not in the DB. Explain why in your README (hint: high write rate, temporary data).

---

## Step 4: Wallet & Transfer Logic

### Endpoints

| Method | Path | Description |
|---|---|---|
| GET | `/wallet/balance` | Return balance (auth required) |
| POST | `/wallet/deposit` | Add funds — body: `{ amount }` |
| POST | `/transactions/send` | Send money — body: `{ toEmail, amount, note? }` |
| GET | `/transactions/history` | Paginated — query: `?page=1&limit=20&type=sent\|received` |

### Transfer — atomic with `SELECT FOR UPDATE`

```
const client = await pool.connect();
try {
  await client.query('BEGIN');

  // Lock the sender's row — prevents concurrent transfers reading stale balance
  const { rows } = await client.query(
    'SELECT balance, frozen FROM wallets WHERE user_id = $1 FOR UPDATE',
    [senderId]
  );
  const wallet = rows[0];

  if (wallet.frozen)          throw new AppError('WALLET_FROZEN', ...403);
  if (wallet.balance < amount) throw new AppError('INSUFFICIENT_FUNDS', ...400);
  if (senderId === receiverId) throw new AppError('SELF_TRANSFER', ...400);

  await client.query('UPDATE wallets SET balance = balance - $1 WHERE user_id = $2', [amount,
senderId]);
  await client.query('UPDATE wallets SET balance = balance + $1 WHERE user_id = $2', [amount,
receiverId]);
  await client.query('INSERT INTO transactions (sender_id, receiver_id, amount, type, status) VALUES
($1,$2,$3,$4,$5)',
    [senderId, receiverId, amount, 'transfer', 'completed']);

  await client.query('COMMIT');
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();
}
```

**Why `SELECT FOR UPDATE` and not just checking the balance?** Without the lock, two concurrent requests can both read balance = $100, both see sufficient funds for $80, and both proceed — resulting in balance = −$60. The database `CHECK (balance >= 0)` will reject the second `UPDATE`, but the first one will have already committed. `FOR UPDATE` forces the second request to wait until the first transaction commits or rolls back.

---

## Step 5: Input Validation

Use `zod` for all request bodies:

```
import { z } from 'zod';

const sendMoneySchema = z.object({
  toEmail: z.string().email(),
  amount: z.number()
    .positive()
    .multipleOf(0.01, 'Amount cannot have more than 2 decimal places')
    .max(10000, 'Single transfer limit is $10,000'),
  note: z.string().max(200).optional(),
});
```

Return `400` with a structured error for any validation failure. Never trust client input.

---

## Step 6: Redis Caching

```
// Cache balance (30s TTL)
await redis.set(`balance:${userId}`, JSON.stringify(balance), 'EX', 30);

// Cache last 20 transactions (60s TTL)
await redis.set(`txHistory:${userId}`, JSON.stringify(txs), 'EX', 60);

// Invalidate on any transaction (both sender and receiver)
await redis.del(`balance:${senderId}`, `txHistory:${senderId}`);
await redis.del(`balance:${receiverId}`, `txHistory:${receiverId}`);
```

Debug endpoint: `GET /debug/cache/:userId` → `{ "balance": "HIT", "txHistory": "MISS" }`

---

## Step 7: Frontend (React)

Pages:

- `/login` — login form, inline error display
- `/register` — registration form
- `/dashboard` — balance card, quick-send form, last 10 transactions
- `/history` — paginated list with Sent / Received filter tabs

Requirements:

- Mobile-first responsive layout (Tailwind breakpoints)
- Loading skeleton while fetching balance/transactions
- Inline errors — never `window.alert()`
- Axios interceptor: redirect to `/login` on any `401`

---

## Step 8: CORS and Security Middleware

```
app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
app.use(express.json({ limit: '100kb' }));
// General rate limit
app.use(generalRateLimiter);   // 100 req / 15 min per IP
// Auth-specific rate limit applied only to /auth/login
app.use('/auth/login', authRateLimiter);  // 10 req / 15 min per IP
```

---

## Milestone Test — Phase 1

`tests/test-phase1.sh` — uses `curl` + `jq` only, exits 0 on full pass:

```
1.  Register User A and User B
2.  Login as A, capture access token
3.  Deposit $100 into A's wallet
4.  Assert A balance == 100.00
5.  Send $40 from A to B
6.  Assert A balance == 60.00
7.  Assert B balance == 40.00
8.  Attempt to send $200 from A → assert 400 INSUFFICIENT_FUNDS
9.  Assert A balance still == 60.00 (no partial deduction)
10. Attempt A to send to A's own email → assert 400 SELF_TRANSFER
11. Call /debug/cache/:userId for A → assert HIT after second balance call
12. POST /auth/login with wrong password 6 times → assert locked response on 6th
13. POST /transactions/send with amount = 12.345 → assert 400 validation error
14. Call /auth/refresh with the httpOnly cookie → assert new access token returned
```

---

## README Requirements — Phase 1

- Why `NUMERIC(15,2)` not `FLOAT` (floating-point representation problem, with an example)
- Why you use HS256 in Phase 1 and why this breaks in Phase 2c
- How `SELECT FOR UPDATE` prevents the double-spend race
- What happens if the server crashes between the two `UPDATE` statements (PostgreSQL guarantees atomicity — rollback happens automatically)
- Why failure counters live in Redis not PostgreSQL (temporary, high-write, TTL-based)
- How to run locally: `docker compose up` then `npm run dev`

---

# Phase 2a — Dockerize the Monolith

**Objective:** Package the Phase 1 application into Docker containers. Everything runs from one `docker compose up` with zero manual steps.

**Estimated Time:** 4–6 hours **Cost:** $0

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| Dockerfile | A recipe for building a container image | docs.docker.com/guides/nodejs |
| Docker Compose | A tool to define and run multi-container apps | docs.docker.com/compose |
| Docker networking | How containers talk to each other by service name | docs.docker.com/network |
| Non-root Docker user | Running as a non-root user inside a container (security) | docs.docker.com/build/building/best-practices |

---

## Step 1: Backend Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build        # tsc → dist/

FROM node:20-alpine AS runner
# Run as non-root — a container process with root privileges can escape to the host
RUN addgroup -S app && adduser -S app -G app
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY --from=builder /app/dist ./dist
COPY migrations ./migrations
USER app
EXPOSE 3000
CMD ["node", "dist/app.js"]
```

The two-stage build keeps the final image small (no TypeScript compiler, no dev dependencies).

## Step 2: Frontend Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build        # Vite → dist/

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

## Step 3: Full Docker Compose

```
version: '3.9'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB:       ${DB_NAME}
      POSTGRES_USER:     ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
      interval: 5s
      timeout: 5s
      retries: 5
```

```
  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
      interval: 5s

  backend:
    build: ./backend
    env_file: .env
    depends_on:
      postgres: { condition: service_healthy }
      redis:    { condition: service_healthy }
    ports:
      - "3000:3000"

  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend

volumes:
  postgres_data:
  redis_data:
```

## Step 4: Database Migration on Startup

The backend must run migrations before accepting requests. On startup:

```
async function runMigrations(pool: Pool) {
  await pool.query(`CREATE TABLE IF NOT EXISTS schema_migrations (version TEXT PRIMARY KEY, applied_at
TIMESTAMPTZ DEFAULT NOW())`);
  const files = fs.readdirSync('./migrations').sort();
  for (const file of files) {
    const version = file.replace('.sql', '');
    const exists = await pool.query('SELECT 1 FROM schema_migrations WHERE version = $1', [version]);
    if (exists.rowCount === 0) {
      const sql = fs.readFileSync(`./migrations/${file}`, 'utf-8');
      await pool.query(sql);
      await pool.query('INSERT INTO schema_migrations (version) VALUES ($1)', [version]);
      console.log(`Applied migration: ${version}`);
    }
  }
}
```

## Milestone Test — Phase 2a

```
docker compose down -v   # clean slate
docker compose up --build -d
```

```
sleep 15                # wait for healthy state
bash tests/test-phase1.sh   # all 14 assertions must still pass
```

If any assertion fails, the app is not correctly containerised — debug before moving on.

---

### README Requirements — Phase 2a

- What the two-stage Docker build achieves (smaller image size, no dev tools in production)
- Why the backend waits for `service_healthy` on postgres before starting
- Why containers run as a non-root user
- `docker compose up --build` quickstart instructions

---

---

# Phase 2b — First Service Split: Auth + App

**Objective:** Separate the application into exactly two services before attempting the full split. Learn inter-service communication with a small, understandable example.

**Estimated Time:** 6–8 hours **Cost:** $0

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| Microservices | Independently deployable services each owning their own data | martinfowler.com/articles/microservices — read the introduction |
| RS256 JWT | Asymmetric JWT: signed with private key, verified with public key. Other services only need the public key. | auth0.com/blog/rs256-vs-hs256 |
| Service-to-service HTTP | One service calling another over the network | No special reading — it is just `fetch()` / `axios` between containers |

**Why upgrade from HS256 to RS256 now?** With two services, both would need the same secret key to verify JWTs — sharing a secret across services is a security risk. With RS256, only `auth-service` holds the private key. Any other service can verify tokens using only the public key.

---

## What You Will Build

| Service | Responsibility | Port |
|---|---|---|
| `auth-service` | Register, login, refresh, logout, JWT issuance | 3001 |
| `app-service` | Wallet, transactions, cache, all business logic | 3002 |
| Nginx | Routes `/auth/*` to auth-service, everything else to app-service | 80 |

Each service has its own PostgreSQL database. `auth-service` owns `users` and `refresh_tokens`. `app-service` owns `wallets` and `transactions`.

---

## Step 1: Upgrade to RS256

Generate a key pair:

```
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem
```

`auth-service` signs tokens with `private.pem`. It exposes `GET /auth/public-key` returning the PEM-encoded public key.

`app-service` fetches the public key at startup and caches it in memory:

```
const { data } = await axios.get('http://auth-service:3001/auth/public-key');
const publicKey = data.publicKey;   // store this in module scope

// On each request:
const payload = jwt.verify(token, publicKey, { algorithms: ['RS256'] });
```

---

## Step 2: Health Check Endpoint (Both Services)

```
app.get('/health', async (req, res) => {
  try {
    await pool.query('SELECT 1');
    res.json({ status: 'ok', service: 'auth-service', db: 'connected', uptime: process.uptime() });
  } catch {
    res.status(503).json({ status: 'error', service: 'auth-service', db: 'disconnected' });
  }
});
```

---

## Step 3: Nginx Gateway

```
server {
  listen 80;
  server_tokens off;    # hide Nginx version from attackers

  location /auth/ {
    proxy_pass http://auth-service:3001;
    proxy_set_header X-Request-ID $request_id;
  }

  location / {
    proxy_pass http://app-service:3002;
    proxy_set_header X-Request-ID $request_id;
  }

  client_max_body_size 1m;
}
```

---

## Milestone Test — Phase 2b

```
1. All 14 Phase 1 test assertions still pass (same flows, now through 2 services + Nginx)
2. Kill auth-service → GET /wallet/balance still works (app-service cached the public key)
3. Kill auth-service → POST /auth/login fails with 502 (auth-service is unavailable)
```

```
4. Restart auth-service → GET /health returns {"status":"ok"} within 30 seconds
5. Send request with expired JWT → 401 returned
```

---

### README Requirements — Phase 2b

- Why you upgraded from HS256 to RS256 at this point (not earlier, not later)
- What `public-key caching` achieves (app-service is not dependent on auth-service being up per-request)
- The database split — why auth-service owns users, not app-service

---

---

# Phase 2c — Full Microservices Split

**Objective:** Complete the service decomposition. Four services, Nginx with rate limiting, full health checks, and Docker networking isolation.

**Estimated Time:** 6–8 hours **Cost:** $0

---

## Before You Start This Phase

You should be comfortable with: Docker Compose, inter-service HTTP, RS256 JWT, health checks. This phase is mostly applying what you learned in 2b to two more services.

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| Docker network isolation | Separate networks for internal vs external traffic | docs.docker.com/network/drivers/bridge |
| Nginx rate limiting | `limit_req_zone` — per-IP request rate enforcement | nginx.org/en/docs/http/ngx_http_limit_req_module |

---

## What You Will Build

| Service | Responsibility | Port |
|---|---|---|
| `user-service` | (renamed from auth-service) Register, login, refresh, public key | 3001 |
| `wallet-service` | Balance, deposit, freeze/unfreeze | 3002 |
| `transaction-service` | Send money, history | 3003 |
| `notification-service` | Log events to its own DB | 3004 |
| Nginx | Gateway with rate limiting | 80 |

### Internal endpoints (not exposed through Nginx)

`wallet-service` and `notification-service` expose `/internal/*` endpoints that are only reachable from within the Docker network, never from the internet:

```
# Block /internal/ at the Nginx gateway
location /internal/ {
  return 403;
}
```

---

## Step 1: Transfer Flow Across Services

`POST /transactions/send` in `transaction-service`:

1. Validate JWT.
2. Call `wallet-service POST /internal/debit` — debit sender.
3. Call `wallet-service POST /internal/credit` — credit receiver.
4. Insert transaction record in `transaction-service` DB.
5. Call `notification-service POST /internal/notify` — fire-and-forget (do not `await`, ignore errors).

**Known limitation — acknowledge in README:** If step 3 succeeds but step 4 crashes, the money moves but no record exists. You will fix this properly in Phase 5a with the saga pattern. For now, your only defence is the `SELECT FOR UPDATE` on wallet-service's debit endpoint.

---

## Step 2: Nginx Rate Limiting

```
limit_req_zone $binary_remote_addr zone=api_limit:10m  rate=60r/m;
limit_req_zone $binary_remote_addr zone=auth_limit:10m rate=5r/m;

location /auth/login {
  limit_req zone=auth_limit burst=5 nodelay;
  proxy_pass http://user_service;
}

location / {
  limit_req zone=api_limit burst=20;
  proxy_pass http://...;
}
```

## Step 3: Docker Network Isolation

```
networks:
  backend-net:      # all services communicate here
  frontend-net:     # only gateway + frontend

services:
  nginx:
    networks: [backend-net, frontend-net]
  user-service:
    networks: [backend-net]
  wallet-service:
    networks: [backend-net]
  # ... etc
```

The frontend container is on `frontend-net` only — it cannot directly reach any backend service, only Nginx.

## Step 4: Correlation IDs

Add `X-Request-ID` propagation:

```
// Middleware in every service
app.use((req, res, next) => {
```

```
  req.requestId = req.headers['x-request-id'] as string || crypto.randomUUID();
  res.setHeader('X-Request-ID', req.requestId);
  next();
});

// When calling another service
axios.post('http://wallet-service:3002/internal/debit', body, {
  headers: { 'X-Request-ID': req.requestId }
});
```

Include `requestId` in every log line. This lets you trace one transfer across all services.

---

### Milestone Test — Phase 2c

```
1. All 14 Phase 1 assertions pass (same flows, now through 4 services + Nginx)
2. Kill notification-service → transfers still succeed (it is non-blocking)
3. Restart notification-service → /health returns ok within 30s
4. GET /internal/debit directly → 403 from Nginx
5. Send 70 requests/min from one IP → 61st returns 429
6. Send request with no JWT → 401 before reaching any service
```

---

### README Requirements — Phase 2c

- Why RS256 matters when you have more than one service verifying tokens
- How Docker network isolation prevents the frontend from bypassing Nginx
- The known limitation in the transfer flow (no transaction record on crash) — what the risk is and when you will fix it
- How correlation IDs help you debug a failing transfer across services

---

---

## Phase 3 — GraphQL API Layer

**Objective:** Add a GraphQL gateway in front of the REST services. Add real-time push subscriptions.

**Estimated Time:** 8–10 hours **Cost:** $0

---

### Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| GraphQL | A query language for APIs — clients request exactly the shape of data they need | graphql.org/learn — first 5 sections |
| Resolvers | Functions that fetch data for each field in a GraphQL schema | apollographql.com/docs/apollo-server/data/resolvers |
| DataLoader | Batches multiple DB/API calls in one event loop tick to solve the N+1 problem | github.com/graphql/dataloader |
| WebSocket | A persistent two-way connection between browser and server | MDN WebSockets |

| | | |
|---|---|---|
| GraphQL subscriptions | Real-time data push over WebSocket using GraphQL | graphql-ws docs |

---

## What You Will Build

A new `graphql-gateway` service. The four REST services from Phase 2c remain unchanged — the gateway calls them via HTTP and adds a GraphQL layer on top.

---

## Step 1: Schema

```
type User {
  id: ID!
  email: String!
  fullName: String!
}

type Wallet {
  balance: Float!
  currency: String!
}

type Transaction {
  id: ID!
  sender: User        # null for deposits
  receiver: User!
  amount: Float!
  type: TransactionType!
  status: TransactionStatus!
  note: String
  createdAt: String!
}

enum TransactionType   { DEPOSIT TRANSFER }
enum TransactionStatus { PENDING COMPLETED FAILED }

type Query {
  me: User!
  myWallet: Wallet!
  transactions(limit: Int = 10, offset: Int = 0, type: TransactionType): [Transaction!]!
}

type Mutation {
  deposit(amount: Float!): Wallet!
  sendMoney(toEmail: String!, amount: Float!, note: String): Transaction!
}

type Subscription {
  transactionReceived: Transaction!
  balanceUpdated: Float!
}
```

---

## Step 2: The N+1 Problem and DataLoader

Without DataLoader, `Query.transactions` for 20 results makes 40 user-service calls (one for `sender`, one for `receiver`, per transaction). This is the N+1 problem.

**Without DataLoader:**

```
GET /transactions → 20 results
→ GET /users/uuid-1   GET /users/uuid-2   (sender + receiver for tx 1)
→ GET /users/uuid-3   GET /users/uuid-4   (sender + receiver for tx 2)
→ ... 40 total calls
```

**With DataLoader:**

```
GET /transactions → 20 results
→ Collect all 40 user IDs within one event loop tick
→ GET /users/batch?ids=uuid-1,uuid-2,...,uuid-40   (1 call)
```

Add `GET /users/batch?ids=id1,id2,id3` to `user-service`.

Create a `UserLoader` — **one per request** (not global — stale data leaks between requests if shared):

```
const createUserLoader = () => new DataLoader<string, User>(async (ids) => {
  const { data } = await axios.get(`http://user-service:3001/users/batch?ids=${ids.join(',')}`);
  return ids.map(id => data[id] || null);
});

// In the request context:
context: () => ({ userLoader: createUserLoader() })
```

---

## Step 3: Subscriptions (in-process PubSub — Phase 3 only)

Use `graphql-subscriptions` `PubSub` for now. When `sendMoney` completes, publish an event:

```
await pubsub.publish(`TRANSACTION_RECEIVED:${receiverId}`, { transactionReceived: tx });
await pubsub.publish(`BALANCE_UPDATED:${senderId}`,  { balanceUpdated: newBalance });
```

**Known limitation — document in your README:** This PubSub is in-memory, inside a single `graphql-gateway` process. If you run two instances of `graphql-gateway`, a subscriber on instance A will not receive events published on instance B. This is a design problem you will solve in Phase 4 by replacing it with MQTT.

---

## Step 4: Production Hardening

These are not optional:

**Disable introspection in production.** Introspection lets anyone query your full schema — field names, types, relationships. Attackers use this to find attack surfaces.

```
plugins: [process.env.NODE_ENV === 'production' ? ApolloServerPluginInlineTraceDisabled() :
undefined],
introspection: process.env.NODE_ENV !== 'production',
```

**Query depth limit** — without this, an attacker sends a deeply nested query that takes seconds to parse:

```
import depthLimit from 'graphql-depth-limit';
```

```
validationRules: [depthLimit(5)]
```

**Query complexity limit** — each field costs points; reject queries over the budget:

```
import { createComplexityRule, simpleEstimator } from 'graphql-query-complexity';
```

**Auth on subscriptions:** Validate JWT during the WebSocket handshake, not per-message:

```
// graphql-ws onConnect handler
onConnect: (ctx) => {
  const token = ctx.connectionParams?.token;
  const payload = jwt.verify(token, publicKey, { algorithms: ['RS256'] });
  return { userId: payload.userId };   // attached to context for all resolvers
}
```

---

## Milestone Test — Phase 3

```
1. sendMoney mutation → assert response has id, amount, status: COMPLETED
2. Open WebSocket subscription as receiver → run sendMoney as sender → assert subscription fires within
2s
3. Query transactions(limit: 20) → check user-service logs → assert 1 batch call, not 40
4. Send any query without JWT → assert { errors: [{ message: "Unauthorized" }] }
5. Send a query nested 7 levels → assert depth limit error, not a server crash
6. Set NODE_ENV=production, send introspection query → assert rejected
```

---

## README Requirements — Phase 3

- The N+1 problem: what it is, why it happens, how DataLoader fixes it
- Why DataLoader must be created per request (not global)
- The in-process PubSub limitation: what breaks with multiple instances
- Why introspection is disabled in production
- WebSocket auth: why validate on connection handshake, not per message

---

---

# Phase 4 — MQTT Event-Driven Architecture

**Objective:** Replace direct HTTP event calls with a message broker. Fix the multi-instance PubSub limitation from Phase 3. Add real-time fraud detection.

**Estimated Time:** 8–10 hours **Cost:** $0 (Mosquitto in Docker)

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| MQTT protocol | Lightweight pub/sub protocol designed for constrained devices and high-throughput events | mqtt.org/getting-started |
| QoS levels | Quality of service: 0 = at most once, 1 = at least once, 2 = exactly once | hivemq.com/blog/mqtt-essentials-part-6-mqtt-quality-of-service-levels |

| | | |
|---|---|---|
| Persistent MQTT session | Broker stores undelivered messages for an offline subscriber | Same link above |
| Redis sorted set | A set where each member has a numeric score — useful for time-window aggregation | redis.io/docs/data-types/sorted-sets |

---

## What You Will Build

- Eclipse Mosquitto broker (in Docker) with password authentication
- Services publish transaction events to MQTT topics instead of calling notification-service directly
- Fraud Detector service monitors all transactions via MQTT subscription
- GraphQL subscriptions now driven by MQTT (fixes Phase 3's multi-instance limitation)

---

## Step 1: Mosquitto With Authentication

`mosquitto/mosquitto.conf`:

```
listener 1883
allow_anonymous false
password_file /mosquitto/config/passwd
persistence true
persistence_location /mosquitto/data/
```

Generate credentials:

```
docker run --rm eclipse-mosquitto:2 mosquitto_passwd -c /tmp/passwd wallet-services
# Prints the hashed credential line — copy to mosquitto/passwd
```

**Why `allow_anonymous false`:** Without credentials, any process that can reach the broker can publish fake fraud alerts or read all user transaction events. Always require authentication, even on internal networks.

---

## Step 2: Topic Structure

Define this before writing any code:

```
wallet/transactions/{userId}/received     # fired when user receives money
wallet/transactions/{userId}/sent         # fired when user sends money
wallet/deposits/{userId}                  # fired on deposit
wallet/alerts/{userId}/low-balance        # fired when balance drops below $10
wallet/system/fraud-suspected             # fired by fraud detector
wallet/system/heartbeat/{service}         # each service publishes every 30s
```

Topic naming rules:

- All lowercase, slash-separated.
- User-specific topics include `{userId}` — subscribers can filter to only their events.
- `wallet/system/` is for infrastructure events.

---

## Step 3: Publishing Events (QoS 1)

After a successful transfer, `transaction-service` publishes:

```
await mqttClient.publishAsync(
  `wallet/transactions/${receiverId}/received`,
  JSON.stringify({
    eventId: crypto.randomUUID(),
    transactionId: tx.id,
    fromEmail: sender.email,
    amount: tx.amount,
    timestamp: new Date().toISOString(),
    requestId: req.requestId,
  }),
  { qos: 1 }   // at-least-once delivery
);
```

**Why QoS 1, not 0:** QoS 0 is fire-and-forget — if the broker is temporarily unreachable, the message is lost silently. For financial events, silent loss is unacceptable. QoS 1 means the broker acknowledges receipt. The consumer must be idempotent (handle duplicate delivery) — which you handle with `eventId`.

**Why not QoS 2:** QoS 2 (exactly-once) requires two round trips per message and adds significant complexity. Since your consumers are already idempotent, QoS 1 is the right balance.

---

## Step 4: Persistent Session Subscriber (Notification Service)

```
const client = mqtt.connect('mqtt://mosquitto:1883', {
  username: 'wallet-services',
  password: process.env.MQTT_PASSWORD,
  clientId: 'notification-service-1',  // fixed ID
  clean: false,                        // persistent session
});

client.subscribe('wallet/#', { qos: 1 });

client.on('message', async (topic, payload) => {
  const event = JSON.parse(payload.toString());

  // Idempotency — skip if already processed
  const exists = await db.query('SELECT 1 FROM notification_log WHERE event_id = $1',
[event.eventId]);
  if (exists.rowCount > 0) return;

  await db.query('INSERT INTO notification_log (event_id, topic, payload) VALUES ($1,$2,$3)',
    [event.eventId, topic, payload.toString()]);
  console.log(JSON.stringify({ level: 'info', msg: 'Event logged', topic, requestId: event.requestId
}));
});
```

Reconnection with exponential backoff:

```
client.on('error', (err) => { /* log */ });
client.on('offline', () => logger.warn('MQTT connection lost, reconnecting...'));
// mqttjs handles reconnection automatically with reconnectPeriod option
```

---

## Step 5: Fraud Detector Service

**Rule:** User sends > $500 in any rolling 60-second window → freeze wallet + publish alert.

**Why Redis sorted set for sliding window:**

- Key: `fraud:window:{userId}`
- Score: timestamp in milliseconds
- Value: `{eventId}:{amount}`

```
const now = Date.now();
const windowStart = now - 60_000;

await redis.zadd(`fraud:window:${userId}`, now, `${event.eventId}:${event.amount}`);
await redis.zremrangebyscore(`fraud:window:${userId}`, 0, windowStart);
await redis.expire(`fraud:window:${userId}`, 120);  // clean up after 2 min inactivity

const entries = await redis.zrangebyscore(`fraud:window:${userId}`, windowStart, '+inf');
const totalSent = entries.reduce((sum, e) => sum + parseFloat(e.split(':')[1]), 0);

if (totalSent > 500) {
  // Publish fraud alert + freeze wallet
}
```

**Why not in-memory:** If fraud-detector restarts, in-memory state is lost. A user could send $490, the service restarts, then send another $490 — no alert fires. Redis persists across restarts.

---

## Step 6: Replace GraphQL PubSub With MQTT

Delete the in-process `PubSub`. Subscribe `graphql-gateway` to:

```
mqttClient.subscribe(`wallet/transactions/${userId}/received`);
mqttClient.on('message', (topic, payload) => {
  const event = JSON.parse(payload.toString());
  // Push to all matching GraphQL WebSocket subscribers
  pubsub.publish('TRANSACTION_RECEIVED', event);
});
```

**This fixes Phase 3's multi-instance problem.** Multiple gateway instances all subscribe to the same MQTT topics. Events published by any service reach all gateway instances, and therefore all connected browser clients.

---

## Milestone Test — Phase 4

```
1. Alice sends Bob $40 → assert MQTT message on wallet/transactions/{bobId}/received
   (use mosquitto_sub CLI tool to listen before sending)
2. Kill notification-service → send a transfer → restart it →
   assert missed event is in notification_log (persistent session replay)
3. Send $600 from Alice in 10 quick transfers (script loop) →
   assert wallet/system/fraud-suspected fires →
   assert Alice's next transfer returns 403 WALLET_FROZEN
4. Open GraphQL subscription as Bob → Alice sends Bob $25 →
   assert subscription fires within 2s
5. Start TWO instances of graphql-gateway on different ports →
   assert subscriber on BOTH instances receives the event
   (this proves MQTT fixed Phase 3's limitation)
```

---

### README Requirements — Phase 4

- Topic naming convention and rationale
- QoS 0 vs 1 vs 2 — why you chose QoS 1
- How persistent sessions deliver missed messages
- The Redis sorted set sliding window algorithm (step by step)
- Why fraud-detector uses Redis not in-memory (restart scenario)
- How MQTT fixes the GraphQL multi-instance PubSub problem

---

---

# Phase 5a — Idempotency & Distributed Locking

**Objective:** Make the transfer API safe against duplicate requests and concurrent races. One concept cluster only.

**Estimated Time:** 6–8 hours **Cost:** $0

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| Idempotency key | A client-generated ID that makes retrying a request safe | stripe.com/blog/idempotency |
| Distributed lock | A lock held in a shared external store (Redis) so multiple processes honour it | redis.io/docs/manual/patterns/distributed-locks |
| SET NX (set-if-not-exists) | Redis atomic operation — the foundation of a single-node distributed lock | Same link above |

---

## Step 1: Idempotency Keys

Every `POST /transactions/send` must include a client-generated `idempotencyKey` (UUIDv4). Requests without one get `400 IDEMPOTENCY_KEY_REQUIRED`.

```
// 1. Fast path: check Redis cache
const cached = await redis.get(`idempotency:${key}`);
if (cached) return JSON.parse(cached);  // return original response immediately

// 2. DB fallback: handles crash-between-execute-and-cache-write
const existing = await db.query(
  'SELECT * FROM transactions WHERE idempotency_key = $1', [key]
);
if (existing.rowCount > 0) {
  const response = formatTransaction(existing.rows[0]);
  await redis.set(`idempotency:${key}`, JSON.stringify(response), 'EX', 86400);
  return response;
}

// 3. Execute the transfer
const tx = await executeTransfer(...);

// 4. Cache the result
```

```
await redis.set(`idempotency:${key}`, JSON.stringify(tx), 'EX', 86400);
return tx;
```

**The crash scenario:** The server executes the transfer (step 3) successfully but crashes before step 4. On retry, the client sends the same key. Step 1 is a MISS (Redis is empty). Step 2 finds the existing transaction in the DB and returns it — no duplicate execution.

---

## Step 2: Distributed Locking With SET NX

**Why `SELECT FOR UPDATE` from Phase 1 is not enough in Phase 2c+:** With separate `wallet-service` and `transaction-service`, the debit and credit calls are two separate HTTP requests. There is no single database transaction wrapping both. Two concurrent transfers from Alice can both call `debit` simultaneously, both see sufficient funds (the lock only applies within a single database connection), and both succeed.

**Solution:** Acquire a Redis lock before the debit call:

```
const lockKey = `wallet-lock:${senderWalletId}`;
const lockId  = crypto.randomUUID();

// SET NX PX: set key if not exists, with millisecond expiry
const acquired = await redis.set(lockKey, lockId, 'NX', 'PX', 5000);
if (!acquired) {
  throw new AppError('LOCK_CONTENTION', 'Another transfer is in progress. Please retry.', 409);
}

try {
  // ... debit, credit, record ...
} finally {
  // Lua script: only delete the key if WE still own it
  // (prevents accidentally releasing another request's lock after TTL expiry)
  await redis.eval(
    `if redis.call("get",KEYS[1])==ARGV[1] then return redis.call("del",KEYS[1]) else return 0 end`,
    1, lockKey, lockId
  );
}
```

**Why not Redlock:** Redlock is a multi-node distributed lock algorithm for when you have 3+ independent Redis instances. With a single Redis instance, `SET NX` is correct and Redlock adds complexity with no safety benefit. Redlock only becomes relevant if you run a Redis Cluster in Phase 6.

**Test:** Write a script that fires 10 concurrent `sendMoney` requests from Alice (same wallet) simultaneously. With locking: exactly the correct number succeed based on balance. Without locking: balance goes negative. Show both runs in your README.

---

## Milestone Test — Phase 5a

```
1. Fire the same idempotency key 5 times simultaneously (parallel curl) →
   assert exactly 1 transaction in DB, all 5 responses are identical
2. Fire 10 concurrent transfers from same wallet simultaneously →
   assert no negative balances
   assert correct number succeeded based on starting balance
3. Kill transaction-service mid-transfer (during the debit/credit window) →
   restart it → assert no orphaned lock in Redis after 5 seconds
   (the TTL cleans it up automatically)
```

---

### README Requirements — Phase 5a

- Why `SELECT FOR UPDATE` stops working across microservices
- The `SET NX` + Lua script pattern and why the Lua script is necessary
- Why Redlock is only needed for multi-node Redis
- The crash-between-execute-and-cache-write scenario and how the DB fallback handles it

---

# Phase 5b — Saga Pattern, Circuit Breaker & Chaos Test

**Objective:** Make multi-step distributed transfers fully compensatable. Handle downstream failures gracefully. Prove everything works under load with a chaos test.

**Estimated Time:** 8–10 hours **Cost:** $0

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| Saga pattern | A sequence of compensatable local transactions that maintain consistency without a distributed lock | microservices.io/patterns/data/saga |
| Circuit breaker | Stops calling a failing service after a threshold, gives it time to recover | martinfowler.com/bliki/CircuitBreaker |
| Chaos engineering | Deliberately breaking systems to find weaknesses before they cause production incidents | principlesofchaos.org |

---

## Step 1: Saga Pattern

A transfer spans three services. If the last step fails, you must undo the earlier ones.

**Compensatable steps:**

| Step | Forward | Compensation |
|---|---|---|
| 1 | Debit sender | Credit sender (refund) |
| 2 | Credit receiver | Debit receiver (clawback) |
| 3 | Record transaction | Mark as `failed` |
| 4 | Publish MQTT event | None (best-effort) |

**Saga log** (append-only — never UPDATE or DELETE rows):

```
CREATE TABLE saga_log (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL,
  step           INT NOT NULL,
  action         TEXT NOT NULL CHECK (action IN ('execute', 'compensate')),
  status         TEXT NOT NULL CHECK (status IN ('success', 'failed')),
  error_detail   TEXT,
```

```
  executed_at    TIMESTAMPTZ DEFAULT NOW()
);
```

**Orchestrator:**

```
execute step 1 → log success
  if fails → log failure, abort (nothing to compensate)

execute step 2 → log success
  if fails → log failure
             compensate step 1 → log compensation
             abort

execute step 3 → log success
  if fails → log failure
             compensate step 2, compensate step 1
             abort

execute step 4 → log warning on failure, continue
```

**Stuck saga detection** — run this periodically or expose as `GET /debug/stuck-sagas`:

```
SELECT transaction_id FROM saga_log
WHERE executed_at < NOW() - INTERVAL '5 minutes'
  AND action = 'execute' AND step = 1 AND status = 'success'
  AND transaction_id NOT IN (
    SELECT transaction_id FROM saga_log
    WHERE action = 'execute' AND step = 3 AND status = 'success'
  );
```

---

## Step 2: Circuit Breaker

```
import CircuitBreaker from 'opossum';

const walletBreaker = new CircuitBreaker(callWalletService, {
  timeout: 3000,
  errorThresholdPercentage: 50,
  resetTimeout: 10000,
  volumeThreshold: 5,   // need at least 5 calls before statistics count
});

walletBreaker.fallback(() => {
  throw new AppError(
    'SERVICE_UNAVAILABLE',
    'Transfer service temporarily unavailable. Your funds have not been moved. Retry in 10 seconds.',
    503
  );
});
```

States: **CLOSED** (normal) → **OPEN** (failing, fallback fires immediately) → **HALF-OPEN** (one probe request) → CLOSED or back to OPEN.

Expose `GET /debug/circuit-status` → `{ "walletService": "CLOSED", "notificationService": "OPEN" }`.

---

## Step 3: Chaos Test

`tests/chaos-test.sh` :

```
#!/usr/bin/env bash
set -e

echo "Seeding 5 users with $1000 each..."
# Register A B C D E, deposit $1000 each
TOTAL=5000.00

echo "Starting 100 random transfers in background..."
for i in $(seq 1 100); do
  # pick random sender/receiver, random $1-$50 amount, unique idempotency key
  curl -s -X POST $BASE_URL/transactions/send ... &
done

sleep 10 && echo "Killing wallet-service..." && docker kill wallet-service-container
sleep 5  && echo "Restarting wallet-service..." && docker start wallet-service-container
sleep 10 && echo "Killing notification-service..." && docker kill notification-service-container

wait   # wait for all background curl calls to finish

sleep 5

ACTUAL=$(psql $WALLET_DB_URL -tAc "SELECT SUM(balance) FROM wallets")
[ "$ACTUAL" = "$TOTAL" ] && echo "[PASS] Money conserved: $ACTUAL" || { echo "[FAIL] Expected $TOTAL
got $ACTUAL"; exit 1; }

NEGATIVE=$(psql $WALLET_DB_URL -tAc "SELECT COUNT(*) FROM wallets WHERE balance < 0")
[ "$NEGATIVE" = "0" ] && echo "[PASS] No negative balances" || { echo "[FAIL] $NEGATIVE negative
wallets"; exit 1; }

STUCK=$(psql $TX_DB_URL -tAc "SELECT COUNT(*) FROM transactions WHERE status = 'pending' AND
created_at < NOW() - INTERVAL '2 minutes'")
[ "$STUCK" = "0" ] && echo "[PASS] No stuck transactions" || { echo "[FAIL] $STUCK stuck"; exit 1; }
```

---

## Milestone Test — Phase 5b

```
1. Run chaos-test.sh → all three [PASS] lines appear
2. Kill wallet-service while a transfer is mid-saga → circuit breaker fires →
   returns 503 with correct message (not 500, not timeout)
3. GET /debug/circuit-status while wallet-service is down → shows OPEN
4. Bring wallet-service back → after 10s GET /debug/circuit-status → shows CLOSED
5. GET /debug/stuck-sagas → empty result
```

---

## README Requirements — Phase 5b

- The saga log is append-only — why you never UPDATE or DELETE rows
- The circuit breaker state machine (include a diagram: CLOSED → OPEN → HALF-OPEN)
- Money conservation as a system invariant — how you verify it with a SQL query
- What "stuck saga" means and what you would do if the query returned results

---

# Phase 6a — First AWS Deployment (Manual)

**Objective:** Get the application running on AWS without Terraform. Understand the components hands-on before automating them.

**Estimated Time:** 6–8 hours **Cost:** $0 (Free Tier) — see breakdown below

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| EC2 | Virtual machine in AWS | aws.amazon.com/ec2/getting-started |
| Security groups | Stateful firewall rules for EC2 instances | docs.aws.amazon.com/vpc/latest/userguide/security-groups |
| IAM | Who can do what in AWS | aws.amazon.com/iam/getting-started |
| RDS | Managed PostgreSQL in AWS | aws.amazon.com/rds/postgresql |
| ECR | Container image registry in AWS | docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli |

**Do this before writing any code:**

1. Create an AWS account.
2. Enable MFA on the root account immediately.
3. Set a billing alarm: CloudWatch → Alarms → Billing → threshold $18 (you get an email before hitting $20).
4. Create an IAM user for your work — never use the root account day-to-day.

---

## Cost Breakdown

| Service | Free Tier | After Free Tier (12 months) |
|---|---|---|
| EC2 t3.micro | 750 hrs/month | ~$8.50/month |
| RDS PostgreSQL t3.micro | 750 hrs/month, 20GB | ~$13/month |
| Redis (Docker on EC2) | Included in EC2 | Included |
| Mosquitto (Docker on EC2) | Included in EC2 | Included |
| ECR | 500MB/month | ~$0.10/month |
| S3 (frontend) | 5GB | ~$0.02/GB |
| Data transfer | 15GB/month | ~$1–2 |
| **Total (Free Tier)** | | **~$1–2/month** |
| **Total (after Free Tier)** | | **~$22–25/month** |

> ***To stay under $20 after Free Tier:*** *Skip RDS. Run PostgreSQL in Docker on the same EC2 instance. Add a daily* `pg_dump` *to S3 (* `aws s3 cp backup.sql s3://your-bucket/` *). Monthly cost drops to ~$10. Trade-off: you lose managed backups and automatic minor version upgrades. Document this decision in your README.*

---

## Step 1: Launch EC2 (Manual — Console)

1. EC2 → Launch Instance
2. AMI: Amazon Linux 2023
3. Instance type: t3.micro (Free Tier eligible)
4. Key pair: create a new one, download the `.pem` file, store it securely — AWS gives it to you once only
5. Security group:
   - Port 80 (HTTP): from anywhere `0.0.0.0/0`
   - Port 443 (HTTPS): from anywhere
   - Port 22 (SSH): from **your IP only** — not `0.0.0.0/0`
6. Storage: increase root volume to 20GB (default 8GB fills quickly with Docker images)

SSH in:

```
chmod 400 your-key.pem
ssh -i your-key.pem ec2-user@your-ec2-public-ip
```

---

## Step 2: Install Docker on EC2

```
sudo yum update -y
sudo yum install -y docker git
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
# Log out and back in for group membership to take effect
```

---

## Step 3: Create RDS (Manual — Console)

1. RDS → Create database → PostgreSQL 15
2. Template: Free Tier
3. Instance: db.t3.micro
4. Storage: 20GB gp2
5. **Public access: No** — RDS should never be reachable from the internet
6. VPC security group: create one that allows port 5432 only from your EC2's security group ID (not from an IP — security group references are more stable)

---

## Step 4: Push Images to ECR

```
# On your local machine:
aws ecr create-repository --repository-name digital-wallet/user-service --region us-east-1
# Repeat for each service

aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
$AWS_ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com

docker build -t digital-wallet/user-service ./services/user-service
docker tag digital-wallet/user-service:latest $ECR_REGISTRY/digital-wallet/user-service:latest
docker push $ECR_REGISTRY/digital-wallet/user-service:latest
```

---

### Step 5: Run on EC2

```
# On EC2:
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
$ECR_REGISTRY
docker compose pull
docker compose up -d
```

Store secrets in environment variables on EC2 — not in a `.env` file committed to git. Set them in `/etc/environment` or pass them via an `.env` file created manually on the instance.

---

### Step 6: HTTPS With Let's Encrypt

```
sudo yum install -y certbot certbot-nginx
sudo certbot --nginx -d yourdomain.com --non-interactive --agree-tos -m you@email.com
```

Certbot auto-renews every 90 days via a cron job. Verify: `sudo certbot renew --dry-run`.

If you have no domain: the EC2 public DNS ( `ec2-1-2-3-4.compute-1.amazonaws.com` ) can be used for HTTP testing. Let's Encrypt will not issue a certificate for it, but that is acceptable for this phase.

---

### Milestone Test — Phase 6a

```
1. SSH into EC2 → docker ps → all services show "Up" and "healthy"
2. curl http://your-ec2-ip/health → returns ok from Nginx
3. Run test-phase1.sh against the live EC2 URL → all assertions pass
4. Run chaos-test.sh against the live EC2 URL → all three [PASS] lines
5. Check your AWS billing dashboard → confirm you are within Free Tier
```

---

### README Requirements — Phase 6a

- Why RDS has no public access (never expose databases directly to the internet)
- Why port 22 is restricted to your IP (SSH brute force is constant on any public IP)
- Your backup strategy for this phase (pg_dump to S3 or accepted risk of RDS managed backups)
- Rollback plan: if you deploy broken code, how do you recover? (re-pull and restart the previous image tag)

---

---

# Phase 6b — Infrastructure as Code & CI/CD

**Objective:** Replace the manual Phase 6a setup with Terraform. Add a GitHub Actions pipeline so every push deploys automatically.

**Estimated Time:** 8–10 hours **Cost:** Same as Phase 6a

---

### Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|

| | | |
|---|---|---|
| Terraform | Declarative infrastructure — describe what you want, Terraform creates it | developer.hashicorp.com/terraform/tutorials/aws-get-started — complete AWS getting started |
| Terraform remote state | Storing `terraform.tfstate` in S3 so it is not lost if your laptop changes | developer.hashicorp.com/terraform/language/settings/backends/s3 |
| SSM Parameter Store | AWS service for storing secrets — free for standard parameters | docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store |
| IAM least privilege | Giving AWS resources only the exact permissions they need | docs.aws.amazon.com/IAM/latest/UserGuide/best-practices |
| GitHub Actions | CI/CD automation triggered by git events | docs.github.com/en/actions |

---

## Step 1: Terraform Remote State (Bootstrap — Manual, Once)

Create these manually before running any `terraform apply`:

```
# S3 bucket for state storage
aws s3api create-bucket --bucket your-name-tf-state --region us-east-1

# Enable versioning (so you can recover from accidental destroy)
aws s3api put-bucket-versioning --bucket your-name-tf-state \
  --versioning-configuration Status=Enabled

# DynamoDB table for state locking (prevents two people running apply simultaneously)
aws dynamodb create-table --table-name terraform-state-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

`terraform/backend.tf`:

```
terraform {
  backend "s3" {
    bucket         = "your-name-tf-state"
    key            = "digital-wallet/prod/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-state-lock"
  }
}
```

**Why this matters:** Without remote state, `terraform.tfstate` lives only on your laptop. If you lose it, Terraform no longer knows what it created and you cannot manage the infrastructure.

---

## Step 2: Terraform Resources

```
terraform/
├── backend.tf
├── main.tf        ← provider, locals
├── variables.tf
├── outputs.tf
├── vpc.tf         ← VPC, subnets, internet gateway, route tables
├── ec2.tf
├── rds.tf
├── s3.tf
├── iam.tf
├── security-groups.tf
└── terraform.tfvars.example
```

**`vpc.tf` — must create a VPC; do not use the default AWS VPC:**

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "digital-wallet-vpc" }
}

resource "aws_subnet" "public" {          # EC2 goes here
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
}

resource "aws_subnet" "private" {         # RDS goes here — no internet access
  vpc_id     = aws_vpc.main.id
  cidr_block = "10.0.2.0/24"
}
```

**`ec2.tf` — use a data source for the AMI, never hardcode:**

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter { name = "name" values = ["al2023-ami-*-x86_64"] }
}

resource "aws_instance" "app" {
  ami                    = data.aws_ami.amazon_linux.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.app.id]
  iam_instance_profile   = aws_iam_instance_profile.ec2.name
  key_name               = var.key_pair_name
  root_block_device { volume_size = 20; volume_type = "gp3" }
  user_data = file("scripts/bootstrap.sh")
  tags = { Name = "digital-wallet-app" }
}
```

`bootstrap.sh` (runs once on first launch — installs Docker, fetches secrets from SSM, starts services).

Subsequent deploys use GitHub Actions SSM Run Command (not UserData — that only runs once).

**`iam.tf` — least privilege for EC2 instance role:**

```
data "aws_iam_policy_document" "ec2_permissions" {
  statement {
    actions   = ["ssm:GetParameter", "ssm:GetParameters"]
    resources = ["arn:aws:ssm:*:*:parameter/digital-wallet/*"]
  }
  statement {
    actions   = ["ecr:GetAuthorizationToken", "ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"]
    resources = ["*"]
  }
  statement {
    actions   = ["logs:CreateLogStream", "logs:PutLogEvents", "logs:CreateLogGroup"]
    resources = ["arn:aws:logs:*:*:log-group:/digital-wallet/*"]
  }
}
```

---

## Step 3: Secrets in SSM Parameter Store

```
aws ssm put-parameter --name "/digital-wallet/prod/db-password"     --value "..." --type SecureString
aws ssm put-parameter --name "/digital-wallet/prod/jwt-private-key" --value "$(cat private.pem)" --
type SecureString
aws ssm put-parameter --name "/digital-wallet/prod/mqtt-password"   --value "..." --type SecureString
```

Fetch in `bootstrap.sh`:

```
DB_PASS=$(aws ssm get-parameter --name /digital-wallet/prod/db-password --with-decryption --query
Parameter.Value --output text)
export DB_PASS
```

---

## Step 4: CI/CD Pipeline

`docker-compose.test.yml` — a minimal compose file for CI (no volumes, in-memory DBs):

```
services:
  postgres-test:
    image: postgres:15-alpine
    environment: { POSTGRES_DB: test, POSTGRES_USER: test, POSTGRES_PASSWORD: test }
  redis-test:
    image: redis:7-alpine
  backend-test:
    build: ./backend
    environment:
      DB_URL: postgresql://test:test@postgres-test/test
      REDIS_URL: redis://redis-test
      JWT_SECRET: test-secret
    depends_on: [postgres-test, redis-test]
```

`.github/workflows/deploy.yml`:

```yaml
name: CI/CD
on:
  push:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run lint && npm run typecheck
        working-directory: services/user-service

  test:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker compose -f docker-compose.test.yml up -d && sleep 10
      - run: bash tests/test-phase1.sh && bash tests/test-phase5a.sh
      - run: docker compose -f docker-compose.test.yml down -v

  build-push:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id:     ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1
      - uses: aws-actions/amazon-ecr-login@v2
      - run: |
          for svc in user-service wallet-service transaction-service notification-service fraud-
detector graphql-gateway; do
            docker build -t ${{ secrets.ECR_REGISTRY }}/$svc:${{ github.sha }} ./services/$svc
            docker push ${{ secrets.ECR_REGISTRY }}/$svc:${{ github.sha }}
          done

  migrate-and-deploy:
    needs: build-push
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id:     ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1
      - name: Run DB migrations first
        run: |
          aws ssm send-command \
            --instance-ids ${{ secrets.EC2_INSTANCE_ID }} \
            --document-name "AWS-RunShellScript" \
            --parameters 'commands=["docker run --rm --env-file /app/.env ${{ secrets.ECR_REGISTRY
}}/migration:${{ github.sha }} npm run migrate"]'
```

```
      - name: Deploy services
        run: |
          aws ssm send-command \
            --instance-ids ${{ secrets.EC2_INSTANCE_ID }} \
            --document-name "AWS-RunShellScript" \
            --parameters 'commands=["cd /app && IMAGE_TAG=${{ github.sha }} docker compose pull &&
docker compose up -d --remove-orphans"]'
```

**Migration before deploy** is the correct order: new code that expects a schema that doesn't exist yet will crash. Always migrate first.

---

## Step 5: CloudWatch Monitoring

| Metric | Source | Alarm |
|---|---|---|
| EC2 CPU | Built-in | > 80% for 5 min → SNS email |
| EC2 memory | CloudWatch Agent | > 85% → SNS email |
| RDS CPU | Built-in | > 70% → SNS email |
| RDS storage | Built-in | < 5GB → SNS email |
| HTTP 5xx count | Nginx log → metric filter | > 5/min → SNS email |
| Billing | Billing (us-east-1) | > $18 → SNS email |

---

## Milestone Test — Phase 6b

```
1. terraform destroy (Phase 6a infra) → terraform apply → everything recreated with zero manual steps
2. Push a commit to main → pipeline runs lint → test → build → migrate → deploy automatically
3. Introduce a failing test assertion → push → pipeline must block at test job, no deploy
4. Revert → push → deploys successfully → HTTPS URL loads and functions
5. Run chaos-test.sh against the live URL → all three [PASS] lines
6. Trigger one CloudWatch alarm deliberately (set threshold to 0%, wait for email)
```

---

## README Requirements — Phase 6b

- Architecture diagram (ASCII or image in `docs/`)
- Why remote Terraform state exists and what happens without it
- Migration-before-deploy order — what breaks if reversed
- Rollback plan: `IMAGE_TAG=previous-sha docker compose up -d`
- Self-hosted Redis/Mosquitto vs ElastiCache/AmazonMQ — cost numbers
- Billing alarm — why it is the first alarm you set up

---

---

# Phase 7a — Kafka, CQRS & Event Sourcing

**Objective:** Replace MQTT with Apache Kafka. Separate reads from writes (CQRS). Make the Kafka log the source of truth (event sourcing).

**Estimated Time:** 12–15 hours **Cost:** Upgrade EC2 to t3.medium (~~$30/month) or t3.small~~ ($15/month) — 3 Kafka brokers need ~3–4 GB RAM. A t3.micro (1GB) cannot run this.

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| Apache Kafka | Distributed, durable, ordered log of events — not a message queue | kafka.apache.org/documentation/#introduction — first 3 sections |
| Consumer group | A set of consumers sharing work across topic partitions | kafka.apache.org/documentation/#intro_consumers |
| KRaft mode | Kafka's built-in consensus (replaces ZooKeeper) | developer.confluent.io/learn/kraft |
| CQRS | Command Query Responsibility Segregation — separate read and write models | martinfowler.com/bliki/CQRS |
| Event sourcing | Storing facts (events) as the primary record, not current state | martinfowler.com/eaaDev/EventSourcing |
| Eventual consistency | The read model may lag behind the write model briefly | cloud.google.com/spanner/docs/true-time-external-consistency |

Spend a day on the reading before writing code. CQRS and event sourcing are architectural patterns — misunderstanding them at the start leads to fundamental structural errors.

---

## What You Will Build

Replace Mosquitto with a 3-broker Kafka cluster. Split services into command (write) and query (read) halves. The Kafka log becomes the canonical source of truth.

---

## Step 1: Kafka in Docker (KRaft Mode — No ZooKeeper)

> ***Why KRaft, not ZooKeeper:*** *ZooKeeper is deprecated since Kafka 3.3 and will be removed. All new Kafka deployments should use KRaft.*

```
kafka-1:
  image: confluentinc/cp-kafka:7.6.0
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka-1:9092
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka-1:9093,2@kafka-2:9093,3@kafka-3:9093
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
    KAFKA_DEFAULT_REPLICATION_FACTOR: 3
    KAFKA_MIN_INSYNC_REPLICAS: 2
    CLUSTER_ID: "Mku3OEVBNTcwNTJENDM2Qk"   # fixed UUID — generate with kafka-storage random-uuid

# kafka-2 and kafka-3: same image, KAFKA_NODE_ID = 2 and 3
```

**Why 3 brokers with `MIN_INSYNC_REPLICAS: 2`:** A write is acknowledged only after 2 of 3 brokers confirm it. You can lose 1 broker without losing data. If 2 brokers die simultaneously, writes are rejected (availability sacrificed for consistency — the right trade for financial data).

---

## Step 2: Topic Design With Retention

```
kafka-topics --create --topic wallet.transactions.events \
  --partitions 6 --replication-factor 3 \
  --config retention.ms=31536000000    # 365 days

kafka-topics --create --topic wallet.wallet.events \
  --partitions 3 --replication-factor 3 \
  --config retention.ms=31536000000

kafka-topics --create --topic wallet.fraud.events \
  --partitions 1 --replication-factor 3 \
  --config retention.ms=7776000000     # 90 days

kafka-topics --create --topic wallet.notifications.dlq \
  --partitions 1 --replication-factor 3 \
  --config retention.ms=604800000      # 7 days
```

**Partition `wallet.transactions.events` by `userId`** (use `userId` as the message key). This ensures all events for one user arrive in order at the same partition.

**Retention policy:** 365 days on transaction events means you can replay the last year of history. After 365 days, old events are deleted. If you need longer retention, archive to S3 before deletion (out of scope — document the limitation).

Create topics explicitly at startup — never rely on auto-create in production.

---

## Step 3: Event Schema With Versioning

```
{
  "eventId":     "uuid",
  "eventType":   "MONEY_TRANSFERRED",
  "version":     1,
  "aggregateId": "transaction-uuid",
  "payload": {
    "senderId":       "uuid",
    "receiverId":     "uuid",
    "amount":         50.00,
    "currency":       "USD",
    "idempotencyKey": "uuid"
  },
  "metadata": {
    "publishedAt":   "2026-09-11T10:00:00Z",
    "service":       "transaction-command-service",
    "correlationId": "uuid"
  }
}
```

**Schema evolution rules — follow these or consumers break silently:**

| Change type | Safe? | How to do it |
|---|---|---|
| Add optional field | ✅ Yes | Add at `version N+1`. Consumers that don't know the field ignore it. |

| | | |
|---|---|---|
| Rename a field | ⚠️ | Add new field at `version N+1`, keep old field, deprecate with a comment. Remove old field only after all consumers updated. |
| Change field type | ❌ No | Never in-place. Add new field, keep old, migrate consumers, then remove old. |
| Remove required field | ❌ No | Make optional first, then remove. |

Consumers must log the `version` of every event they process. This lets you detect when an old consumer is processing events it was not designed for.

---

## Step 4: CQRS Split

**Write side (Command Services):**

- `transaction-command-service` — accepts `POST /transactions/send`, validates, publishes `MONEY_TRANSFERRED` event, returns `202 Accepted`
- `wallet-command-service` — accepts `POST /wallet/deposit`, publishes `FUNDS_DEPOSITED` event

> ***Why 202 and not 200:*** *The DB has not been written yet when the command service returns. The event is published to Kafka; the query service will process it and update the read model shortly. Returning 200 would be a lie.*

**Read side (Query Services):**

- `transaction-query-service` — consumes `wallet.transactions.events`, materialises into a PostgreSQL read model, serves `GET /transactions`
- `wallet-query-service` — consumes `wallet.wallet.events`, materialises current balance, serves `GET /wallet/balance`

**Rule:** Query services have no write endpoints. Command services have no read endpoints (except health checks).

**Handling eventual consistency in the UI:** After `sendMoney` returns 202, the balance on `GET /wallet/balance` may briefly show the old value (the consumer has not processed the event yet). The frontend must:

1. Show a "processing" indicator.
2. Poll `/wallet/balance` every 500ms for up to 3 seconds until the balance changes.
3. Explain this in the README — it is not a bug, it is the fundamental trade-off of CQRS.

---

## Step 5: Consumer Lag Monitoring

Add `GET /debug/consumer-lag` to `wallet-query-service`:

```
// Use the Kafka admin client to fetch consumer group offsets
// Compare to the latest offset per partition
// Return lag per partition
```

A growing lag means the consumer cannot keep up. A lag of 0 at steady state is the target.

---

## Step 6: Event Sourcing — Replay Test

`scripts/replay.sh`:

1. Spin up a blank PostgreSQL instance.
2. Consume every event from `wallet.transactions.events` starting at offset 0.
3. Apply each event to rebuild balances and transaction records from scratch.
4. Compare final balances against the live production DB.

```
Replaying 4821 events from offset 0...
[PASS] All 5 wallet balances match production.
[PASS] All 4821 transactions match production.
```

This proves the Kafka log is self-contained and the PostgreSQL DB is just a projection of it.

---

## Milestone Test — Phase 7a

```
1. replay.sh → all balances match production
2. Kill one Kafka broker → continue sending transfers →
   assert all succeed (MIN_INSYNC_REPLICAS=2 + replication=3 tolerates 1 broker loss)
3. GET /debug/consumer-lag → shows 0 at steady state after all transfers complete
4. Send a transfer (202 returned) → poll /wallet/balance → assert it updates within 2 seconds
5. Inspect a consumed event with version=1 → add a new optional field to the schema (version=2) →
   assert old consumers still process version=2 events without errors
```

---

## README Requirements — Phase 7a

- CQRS: commands vs queries, why they are separated
- Why the command service returns 202 not 200
- Eventual consistency: what it means for the UI, how the frontend handles it
- Event sourcing: what "the Kafka log is the source of truth" means
- Kafka retention policy: what replay guarantees you have and what you lose after 365 days
- Schema evolution rules: why backward compatibility matters and how to maintain it
- Why you use KRaft not ZooKeeper

---

---

# Phase 7b — Security Hardening & Penetration Testing

**Objective:** Add mutual TLS between all internal services. Harden against OWASP Top 10. Add a tamper-evident audit log. Run a penetration test.

**Estimated Time:** 10–12 hours **Cost:** +$0

---

## Before You Start This Phase

**New concepts this phase introduces:**

| Concept | What It Is | Where to Read |
|---|---|---|
| mTLS (mutual TLS) | Both client and server present certificates — not just server-to-client | cloudflare.com/learning/access-management/what-is-mutual-tls |
| OWASP Top 10 | The 10 most critical web security risks, updated every 4 years | owasp.org/www-project-top-ten — read all 10 |
| Certificate Authority (CA) | A trusted entity that signs certificates, proving identity | letsencrypt.org/docs/a-warm-welcome-to-asn1-and-der (optional deep dive) |
| Cryptographic hash chain | Each record includes a hash of the previous record — tampering breaks the chain | en.wikipedia.org/wiki/Hash_chain |
| OWASP ZAP | Free, open-source penetration testing tool | owasp.org/www-project-zap |

Read the OWASP Top 10 page completely before writing any code in this phase.

---

## Step 1: mTLS Between Internal Services

Generate a self-signed Certificate Authority and per-service certificates:

```
# Internal CA
openssl genrsa -out certs/ca.key 4096
openssl req -new -x509 -days 365 -key certs/ca.key -out certs/ca.crt \
  -subj "/CN=digital-wallet-internal-ca"

# One certificate per service
for svc in user wallet transaction notification fraud graphql; do
  openssl genrsa -out certs/$svc.key 2048
  openssl req -new -key certs/$svc.key -out certs/$svc.csr -subj "/CN=$svc-service"
  openssl x509 -req -days 365 -in certs/$svc.csr -CA certs/ca.crt \
    -CAkey certs/ca.key -CAcreateserial -out certs/$svc.crt
done
```

**What "mutual" means:** In standard TLS, only the server presents a certificate (proving it is who it says it is). The client is anonymous. In mTLS, the client also presents a certificate. The server checks that the client cert is signed by the trusted CA — so a rogue process cannot impersonate `wallet-service` even if it is inside the Docker network.

Each service's HTTPS server:

```
https.createServer({
  key:                fs.readFileSync(`certs/${serviceName}.key`),
  cert:               fs.readFileSync(`certs/${serviceName}.crt`),
  ca:                 fs.readFileSync('certs/ca.crt'),
  requestCert:        true,
  rejectUnauthorized: true,
}, app);
```

Each service making outbound calls:

```
axios.create({
  httpsAgent: new https.Agent({
    key:  fs.readFileSync(`certs/${serviceName}.key`),
    cert: fs.readFileSync(`certs/${serviceName}.crt`),
    ca:   fs.readFileSync('certs/ca.crt'),
  })
});
```

**Certificate rotation ( `scripts/rotate-certs.sh` ):** Regenerate all service certs (keep the CA). Perform a rolling restart: one service at a time, wait for its health check to return `ok` , then proceed to the next. Log each step. If any service fails to come healthy within 60 seconds, halt and alert.

---

## Step 2: OWASP Top 10 Hardening

Work through each item. Every mitigation implemented. Every decision documented.

| # | Risk | Your Mitigation |
|---|---|---|

| | | |
|---|---|---|
| A01 | Broken Access Control | JWT on every endpoint; users can only access their own wallet; `/internal/*` blocked at Nginx; no IDOR possible (UUIDs, not sequential IDs) |
| A02 | Cryptographic Failures | bcrypt cost 12; RS256 JWT; TLS 1.2+ only; no secrets in logs; no sensitive fields in API responses |
| A03 | Injection | Parameterised queries everywhere (`$1, $2, ...`); `zod` validates all inputs; no string-concatenated SQL |
| A04 | Insecure Design | Saga pattern (no partial state); idempotency keys (no duplicates); rate limiting at gateway; self-transfer blocked |
| A05 | Security Misconfiguration | No default credentials; `server_tokens off`; Docker non-root users; only ports 80, 443, 22 (your IP) open on EC2 |
| A06 | Vulnerable Components | `npm audit --audit-level=high` in every CI run; `trivy image` scan in CI; block build on High or Critical CVEs |
| A07 | Auth Failures | 15-min access tokens; refresh token stored as hash; refresh rotation on use; account lockout; brute-force rate limit |
| A08 | Software Integrity | Pin Docker image digests in Compose (`image: postgres@sha256:...`) |
| A09 | Logging Failures | All auth events and transfers in audit log; logs shipped to CloudWatch; no PII or secrets in logs |
| A10 | SSRF | No user-supplied URLs fetched server-side; Nginx blocks requests to `169.254.0.0/16` (AWS metadata service) |

---

## Step 3: Security Headers

```
add_header Strict-Transport-Security  "max-age=31536000; includeSubDomains; preload" always;
add_header X-Frame-Options            "DENY" always;
add_header X-Content-Type-Options     "nosniff" always;
add_header Referrer-Policy            "strict-origin-when-cross-origin" always;
add_header Content-Security-Policy    "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline';" always;
add_header Permissions-Policy         "geolocation=(), microphone=(), camera=()" always;
ssl_protocols                         TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers             on;
```

Check with [securityheaders.com](securityheaders.com) — target grade A.

---

## Step 4: Tamper-Evident Audit Log

```
CREATE TABLE audit_log (
  id           BIGSERIAL PRIMARY KEY,
  event_type   TEXT NOT NULL,
  actor_id     UUID,
  target_id    UUID,
  payload      JSONB NOT NULL,
  ip_address   INET,
  occurred_at  TIMESTAMPTZ DEFAULT NOW(),
  prev_hash    TEXT,         -- NULL for row id=1 (see GENESIS handling below)
```

```
  row_hash     TEXT NOT NULL
);

-- Application DB user cannot update or delete audit rows
REVOKE UPDATE, DELETE ON audit_log FROM wallet_app_user;
```

**Hash formula:**

```
row_hash = SHA256(
  id::text
  || event_type
  || COALESCE(actor_id::text, '')
  || COALESCE(target_id::text, '')
  || payload::text
  || occurred_at::text
  || COALESCE(prev_hash, 'GENESIS')   ← row 1 has no prev_hash, use literal 'GENESIS'
)
```

**The GENESIS edge case:** The very first row ( `id = 1` ) has no previous row, so `prev_hash` is `NULL` . Using `NULL` directly in the formula is fragile (SQL null propagation). Using the literal string `'GENESIS'` as the sentinel is the correct approach. Document this explicitly — it is a common source of broken hash chains.

**Events that must be audited:**

- User registration
- Login (success and failure) — include IP address
- JWT refresh
- Every money transfer (including failed attempts) — include idempotency key
- Wallet freeze / unfreeze
- Fraud alert triggered
- Admin actions (if any)

**Verification script** `scripts/verify-audit-log.sh` **:** Walk every row from `id=1` , recompute the hash, compare to `row_hash` . If any mismatch: `[FAIL] Tamper detected at row {id}` . If all match: `[PASS] Audit log integrity verified — N events` .

---

## Step 5: Penetration Test With OWASP ZAP

```
docker run --rm -v $(pwd)/security:/zap/wrk owasp/zap2docker-stable \
  zap-full-scan.py -t https://your-staging-url.com -r zap-report.html
```

**Pass criteria:**

- Zero High severity findings.
- For every Medium severity finding: document in README whether it is a real vulnerability you fixed or a false positive with justification.
- Commit the final report to `security/zap-report.html` .

Common findings to expect and fix: missing security headers (fixed in Step 3), weak TLS configuration, exposed stack traces in error responses, missing CSP.

---

## Milestone Test — Phase 7b

```
1. Attempt service-to-service HTTP call without client certificate →
   rejected with TLS handshake error (not a 401, not a 403 — a TLS error before HTTP)
```

```
2. verify-audit-log.sh → [PASS]
3. UPDATE audit_log SET event_type='TAMPERED' WHERE id=50 →
   verify-audit-log.sh → [FAIL] Tamper detected at row 50
4. UPDATE audit_log SET event_type='TAMPERED' WHERE id=1 (GENESIS row) →
   verify-audit-log.sh → [FAIL] Tamper detected at row 1
5. Run rotate-certs.sh → all services come back healthy, mTLS still works
6. ZAP report → zero High severity findings
7. securityheaders.com → grade A
```

---

### README Requirements — Phase 7b

- mTLS: what "mutual" means vs standard TLS, why you added it
- Each OWASP item: what the risk is and exactly how you mitigated it
- The GENESIS sentinel in the audit log hash chain — why `NULL` is wrong
- Each ZAP finding: vulnerability or false positive, and what you did

---

---

# Final Deliverables

By the end of all ten phases, you must have:

- [ ] Clean git history — one commit per logical feature, meaningful messages
- [ ] Root `README.md` linking to each phase's documentation
- [ ] `docs/learning-log.md` — your Phase 0 answers, updated with lessons learned per phase
- [ ] `tests/` — all milestone test scripts (test-phase1.sh, test-phase2c.sh, test-phase5a.sh, chaos-test.sh, replay.sh, verify-audit-log.sh, rotate-certs.sh)
- [ ] `terraform/` — full IaC for AWS deployment, remote state configured
- [ ] `.github/workflows/deploy.yml` — full CI/CD pipeline including docker-compose.test.yml
- [ ] `security/zap-report.html` — final penetration test report
- [ ] `docs/architecture-phase6.png` and `docs/architecture-phase7.png` — architecture diagrams
- [ ] `openapi.yaml` — REST API documentation

---

# Skills Map

| Phase | Core Skills Introduced |
|---|---|
| 0 | HTTP, SQL, TypeScript, JWT basics — confirmed via written questions |
| 1 | REST API, PostgreSQL (transactions, triggers, constraints), Redis, bcrypt, HS256 JWT, refresh tokens, brute-force protection, zod validation |
| 2a | Docker multi-stage builds, Docker Compose, container health checks, non-root users |
| 2b | RS256 JWT (asymmetric), public key distribution, 2-service communication, Nginx routing |
| 2c | Full microservice split, Docker network isolation, Nginx rate limiting, correlation IDs |
| 3 | GraphQL schema, resolvers, DataLoader (N+1), WebSocket subscriptions, introspection security |
| 4 | MQTT protocol, QoS levels, persistent sessions, topic design, sliding window fraud detection |
| 5a | Idempotency keys, SET NX distributed lock, Lua atomic scripts, crash scenarios |
| 5b | Saga pattern, saga log, circuit breaker (3 states), chaos engineering, money conservation |

| | |
|---|---|
| 6a | EC2, RDS, security groups, IAM, ECR, manual AWS deployment, Let's Encrypt |
| 6b | Terraform (VPC, IaC), remote state, SSM secrets, GitHub Actions CI/CD, CloudWatch |
| 7a | Apache Kafka (KRaft), CQRS, event sourcing, schema evolution, consumer lag, eventual consistency |
| 7b | mTLS, OWASP Top 10, security headers, tamper-evident audit log, OWASP ZAP penetration test |

---

*Start from Phase 0. Each phase is designed so that completing it gives you everything you need for the next one — and nothing more.*